```
    std::shared_ptr<T> wait_and_pop()
    {
        std::unique_lock<std::mutex> lk(mut);
        data_cond.wait(lk,[this]{return !data_queue.empty();});
        std::shared_ptr<T> res=data_queue.front();    <-3
        data_queue.pop();
        return res;
    }
    std::shared_ptr<T> try_pop()
    {
        std::lock_guard<std::mutex> lk(mut);
        if(data_queue.empty())
            return std::shared_ptr<T>();
        std::shared_ptr<T> res=data_queue.front();    <-4
        data_queue.pop();
        return res;
    }

    void push(T new_value)
    {
        std::shared_ptr<T> data(
            std::make_shared<T>(std::move(new_value)));    <-5
        std::lock_guard<std::mutex> lk(mut);
        data_queue.push(data);
        data_cond.notify_one();
    }

    bool empty() const
    {
        std::lock_guard<std::mutex> lk(mut);
        return data_queue.empty();
    }
};
```

The basic consequences of holding the data by `std::shared_ptr<>` are straightforward: the pop functions that take a reference to a variable to receive the new value now have to dereference the stored pointer ❶, ❷, and the pop functions that return a `std::shared_ptr<>` instance can just retrieve it from the queue ❸, ❹ before returning it to the caller.

If the data is held by `std::shared_ptr<>`, there's an additional benefit: the allocation of the new instance can now be done outside the lock in `push()` ❺, whereas in listing 6.2 it had to be done while holding the